

Universidade do Algarve
Faculdade de Ciências e Tecnologia

Laboratório de Engenharia de Software
Ano letivo 2025/2026

Spot-On

Sistema de Gestão de Espaços de Estudo

Relatório Técnico do Trabalho Prático

Docentes: José Barateiro
Marco Giunti

Equipa: André Martins – n.º a35463
António Matoso – n.º a84100
Tomás Machado – n.º a84014
Vasco Hilário – n.º 83993

22 de maio de 2026

Conteúdo

1	Introdução	2
2	Visão Geral do Sistema	2
2.1	Funcionalidades Principais	2
3	Atores do Sistema	3
4	Requisitos	3
4.1	Requisitos Funcionais	3
4.2	Requisitos de Qualidade	4
5	Arquitetura do Sistema	4
5.1	Visão de Alto Nível	4
5.2	Camadas Lógicas	4
6	Stack Tecnológico	5
6.1	Justificação Detalhada das Escolhas-Chave	5
6.2	Estratégia de Autenticação: do SSO ao <i>Passwordless</i>	5
7	Modelo de Dados	7
7.1	Entidades de Domínio	8
7.2	Diagrama do Modelo de Dados	8
7.3	Decisões de Modelação Relevantes	8
7.4	Excerto do <i>Schema</i> Prisma	9
8	Backend e API	9
9	Controlo de Versões	9
10	Gestão de Projeto – SCRUM	9
11	Testes e Integração Contínua	10
11.1	Testes Automatizados	10
11.2	Pipeline de CI	10
12	Deployment	10
12.1	Execução Local	10
12.2	Ambiente de Produção	11
13	Conclusão	11

1 Introdução

O presente relatório documenta o trabalho prático desenvolvido no âmbito da unidade curricular de Laboratório de Engenharia de Software, no ano letivo 2025/2026. O projeto, denominado **Spot-On**, consiste num sistema de gestão de espaços de estudo, concebido para o contexto da biblioteca da Universidade do Algarve.

A motivação para o tema surge de uma necessidade frequente entre estudantes: identificar rapidamente que mesas individuais ou salas de grupo se encontram disponíveis num determinado momento, sem percorrer fisicamente todos os pisos da biblioteca. O Spot-On disponibiliza essa informação em tempo real através de uma interface visual baseada nas plantas dos pisos, permitindo ainda organizar sessões de estudo, convidar colegas, e gerir a ocupação dos espaços com recurso a códigos QR.

Este documento descreve a visão geral do sistema, os atores envolvidos, os requisitos funcionais e de qualidade, a arquitetura e o modelo de dados adotados, bem como a justificação das principais decisões técnicas.

2 Visão Geral do Sistema

O Spot-On é uma aplicação web que permite a alunos da Universidade do Algarve visualizar e gerir o uso de espaços de estudo na biblioteca. O sistema disponibiliza uma representação visual dos pisos da biblioteca, sobre a qual são marcados os espaços (mesas individuais e salas de grupo), com indicação do seu estado de ocupação em tempo real.

2.1 Funcionalidades Principais

- **Autenticação institucional** – acesso restrito a contas de correio @ualg.pt, com autenticação *passwordless* via *magic link* enviado por email;
- **Visualização de plantas** – exibição interativa das plantas dos pisos da biblioteca, com sobreposição dos espaços de estudo;
- **Gestão de espaços** – mesas individuais e salas de grupo, com atributos como capacidade, presença de tomada elétrica, computador ou quadro interativo;
- **Sessões de estudo** – criação, consulta e encerramento de sessões, associadas a um espaço e a um anfitrião (*host*);
- **Confirmação de presença via QR code** – cada espaço dispõe de um *token* QR rotativo que valida fisicamente a ocupação;
- **Convites e participações** – o anfitrião pode convidar outros utilizadores para a sessão; estes confirmam ou rejeitam o convite;
- **Reports** – mecanismo para reportar comportamentos incorretos, com confirmação por outros utilizadores e janela temporal limitada;
- **Notificações** – sistema in-app para pedidos de adesão, pedidos de prova de presença e outros eventos relevantes;
- **Gamificação** – atribuição automática de pontos e *badges* (mensais, anuais ou permanentes) que premeiam a utilização regular e correta do sistema;
- **Administração** – gestão de plantas, espaços, *badges* e moderação de utilizadores (incluindo banimento temporário).

3 Atores do Sistema

Foram identificados dois atores humanos no sistema:

Ator	Descrição
Aluno	Utilizador final do sistema. Pode autenticar-se, consultar a disponibilidade de espaços, criar e gerir sessões de estudo (anfitrião), participar em sessões de outros (convidado), submeter <i>reports</i> , receber notificações e acumular pontos e <i>badges</i> .
Administrador	Responsável pela gestão da configuração do sistema. Define plantas dos pisos, cria e edita espaços, configura <i>badges</i> e modera utilizadores.

Tabela 1: Atores do sistema Spot-On.

4 Requisitos

4.1 Requisitos Funcionais

A tabela seguinte sumariza os requisitos funcionais identificados, agrupados por área. A lista não é exaustiva, mas reflete o conjunto efetivamente implementado.

ID	Área	Descrição
RF-01	Autenticação	O sistema autentica utilizadores via <i>magic link</i> enviado para o seu email institucional.
RF-02	Autenticação	O sistema rejeita utilizadores cujo email não pertença ao domínio @ualg.pt.
RF-03	Espaços	O sistema apresenta as plantas dos pisos com a localização dos espaços.
RF-04	Espaços	O sistema indica em tempo real o estado de ocupação de cada espaço.
RF-05	Espaços	O utilizador pode filtrar espaços por características (capacidade, tomada, computador, quadro).
RF-06	Sessões	Um utilizador pode iniciar uma sessão de estudo num espaço disponível.
RF-07	Sessões	Um anfitrião pode convidar outros utilizadores para a sessão.
RF-08	Sessões	Um utilizador pode aceitar ou rejeitar um convite recebido.
RF-09	Sessões	A presença é confirmada por leitura do QR code do espaço.
RF-10	Reports	Um utilizador pode submeter um <i>report</i> sobre uma sessão.
RF-11	Reports	Outros utilizadores podem confirmar um <i>report</i> , num prazo limitado.
RF-12	Notificações	O sistema envia notificações in-app para eventos relevantes.
RF-13	Gamificação	O sistema atribui pontos e <i>badges</i> automaticamente.
RF-14	Admin	O administrador gere plantas, espaços, <i>badges</i> e utilizadores.
RF-15	Admin	O administrador pode banir temporariamente um utilizador.

Tabela 2: Requisitos funcionais do Spot-On.

4.2 Requisitos de Qualidade

Os requisitos de qualidade (não funcionais) descrevem propriedades transversais ao sistema, não associadas a funcionalidades específicas. A tabela seguinte enumera os requisitos identificados, agrupados por categoria ISO/IEC 25010, com critérios verificáveis sempre que possível.

ID	Categoria	Descrição e Critério
RQ-01	Segurança	O acesso ao sistema é restrito a endereços de email do domínio <code>@ualg.pt</code> ; tentativas com outros domínios são rejeitadas no <i>backend</i> .
RQ-02	Segurança	Os <i>tokens</i> de <i>magic link</i> expiram em 5 minutos e só podem ser utilizados uma vez.
RQ-03	Segurança	O <i>token</i> QR de cada espaço é rotacionado periodicamente, impedindo a reutilização de fotografias antigas para validar presença.
RQ-04	Segurança	Utilizadores banidos (<code>bannedUntil</code> no futuro) não conseguem iniciar sessão nem participar em sessões.
RQ-05	Manutenibilidade	A camada de persistência é totalmente acedida através de um ORM, isolando o restante código de detalhes do SGBD.
RQ-06	Manutenibilidade	Todas as alterações ao modelo de dados são versionadas como migrações no repositório (<code>prisma/migrations</code>).
RQ-07	Portabilidade	A aplicação executa em qualquer máquina com Docker instalado, através de um único comando (<code>docker compose up</code>), sem dependências locais adicionais.
RQ-08	Fiabilidade	A <i>pipeline</i> de CI executa, em cada <i>push</i> e <i>pull request</i> , análise estática (<i>lint</i>), testes automatizados e <i>build</i> de produção; nenhum <i>merge</i> em <code>main</code> ocorre sem que estes passos passem.
RQ-09	Usabilidade	A consulta de disponibilidade é feita visualmente sobre a planta do piso, sem necessidade de o utilizador conhecer identificadores de espaços.
RQ-10	Compatibilidade	A aplicação é responsiva e funciona em navegadores modernos (<i>desktop</i> e <i>mobile</i>), incluindo os tablets fixados nos espaços para exibir o QR code.

Tabela 3: Requisitos de qualidade do Spot-On.

5 Arquitetura do Sistema

5.1 Visão de Alto Nível

O Spot-On segue uma arquitetura **cliente-servidor** com aplicação web *full-stack* construída sobre Next.js. O *frontend* (React) e o *backend* (API *routes* do Next.js) coabitam no mesmo projeto, mas estão claramente separados em termos de responsabilidades. A persistência é assegurada por uma base de dados PostgreSQL, acedida exclusivamente através do ORM Prisma. A autenticação é delegada à biblioteca NextAuth.js, que gere o fluxo de *magic link* sobre o domínio institucional.

[Diagrama de arquitetura a inserir]

5.2 Camadas Lógicas

Internamente, a aplicação Next.js encontra-se organizada em três camadas:

- Apresentação** – componentes React responsáveis pela interface, incluindo a renderização das plantas, formulários e *dashboards*. Faz uso de *Server Components* e *Client Components* consoante a necessidade de interatividade.
- API e regras de negócio** – API *routes* do Next.js, onde reside a validação de dados, regras de domínio (e.g., elegibilidade para iniciar sessão, validade do QR *token*, atribuição de pontos) e a orquestração das operações.

3. **Persistência** – camada de acesso a dados, totalmente delegada ao Prisma, que gera um cliente tipado a partir do *schema* `schema.prisma`.

6 Stack Tecnológico

A tabela seguinte resume as tecnologias adotadas e as principais alternativas ponderadas.

6.1 Justificação Detalhada das Escolhas-Chave

Next.js vs. React + Express. Considerou-se manter *frontend* e *backend* em projetos independentes (React no *client-side* e Express como API). Optou-se por Next.js pelos seguintes motivos: (i) elimina a necessidade de gerir *cross-origin requests* entre dois servidores; (ii) permite tipagem partilhada entre cliente e servidor sem etapas adicionais de geração; (iii) o seu modelo de *Server Components* simplifica o carregamento inicial de páginas como a planta da biblioteca; e (iv) reduz a complexidade de *deployment*.

Prisma vs. TypeORM. Prisma destaca-se pelo seu *schema* declarativo (`schema.prisma`), que serve simultaneamente de fonte de verdade para a base de dados e gerador do cliente tipado. As migrações geram-se automaticamente a partir de alterações ao *schema* (`prisma migrate dev`), tornando o ciclo de evolução do modelo mais rápido e menos propenso a erros do que abordagens baseadas em decoradores ou em SQL puro. Em projetos de média dimensão como este, a curva de aprendizagem é igualmente menor.

PostgreSQL vs. MySQL. O modelo do Spot-On é fortemente relacional (sessões pertencem a espaços, participações são tabelas-pivô, *reports* ligam-se a sessões e a utilizadores) e faz uso intensivo de tipos enumerados (`SessionStatus`, `ReportStatus`, `NotificationType`). PostgreSQL suporta nativamente `enum types` e oferece um conjunto mais rico de funções temporais (necessárias para janelas de confirmação de *reports*, p. ex. `now() + interval '10 minutes'`).

6.2 Estratégia de Autenticação: do SSO ao *Passwordless*

A gestão de identidade foi implementada com o NextAuth.js (e o `@next-auth/prisma-adapter`). A arquitetura de autenticação sofreu uma iteração importante durante o desenvolvimento, adaptando-se às restrições da infraestrutura da universidade.

Tentativa inicial: SSO via Microsoft Entra ID. O planeamento inicial previa um verdadeiro *Single Sign-On* (SSO) com o *provider* do Azure AD no NextAuth. Como a comunidade académica utiliza o ecossistema Microsoft (Outlook/Office 365), o objetivo era um login direto e fluido (“Continuar com a conta da Universidade”).

Bloqueio técnico: restrições de *tenant*. Durante a implementação, a equipa deparou-se com um erro 401 (`Unauthorized`) no portal do Azure (Figura 1). Concluiu-se que as políticas de segurança dos Serviços de Informática da UAlg bloqueiam ativamente o registo de novas aplicações (*App Registrations*) por parte dos alunos no *tenant* principal da universidade.

Camada	Tecnologia	Alternativa	Justificação
<i>Framework full-stack</i>	Next.js (React)	React + Express separados	Unifica UI e API num só projeto, simplifica <i>deployment</i> e tipagem partilhada.
Linguagem (<i>frontend</i> e <i>backend</i>)	TypeScript	JavaScript	Mesma linguagem em ambas as vertentes, com tipagem estática que reduz erros e integra-se com o cliente Prisma gerado.
ORM	Prisma	TypeORM, Sequelize	<i>Schema</i> declarativo, migrações automáticas, tipagem forte ponta-a-ponta.
Base de dados	PostgreSQL	MySQL, MongoDB	Modelo relacional adequado às entidades; enums nativos; maturidade.
Autenticação	NextAuth.js (<i>magic link</i>)	Auth0, Clerk, implementação própria	Integração nativa com Next.js, fluxo <i>passwordless</i> pronto a usar.
Containerização	Docker + Docker Compose	VM, deploy direto	<i>Setup</i> reproduzível em qualquer máquina com um único comando.
<i>Hosting</i>	Railway	Vercel, AWS, Render	<i>Deploy</i> simples a partir de Git, base de dados PostgreSQL gerida incluída.
Testes	Jest	Vitest, Mocha	Padrão consolidado em projetos React/Next.js, <i>tooling</i> maduro.
CI	GitHub Actions	GitLab CI, CircleCI	Integração nativa com o repositório, <i>free tier</i> suficiente.
Gestão	Jira	Trello, ClickUp	Suporte completo a SCRUM (<i>backlog</i> , <i>sprints</i> , <i>boards</i>).

Tabela 4: Stack tecnológico e alternativas consideradas.

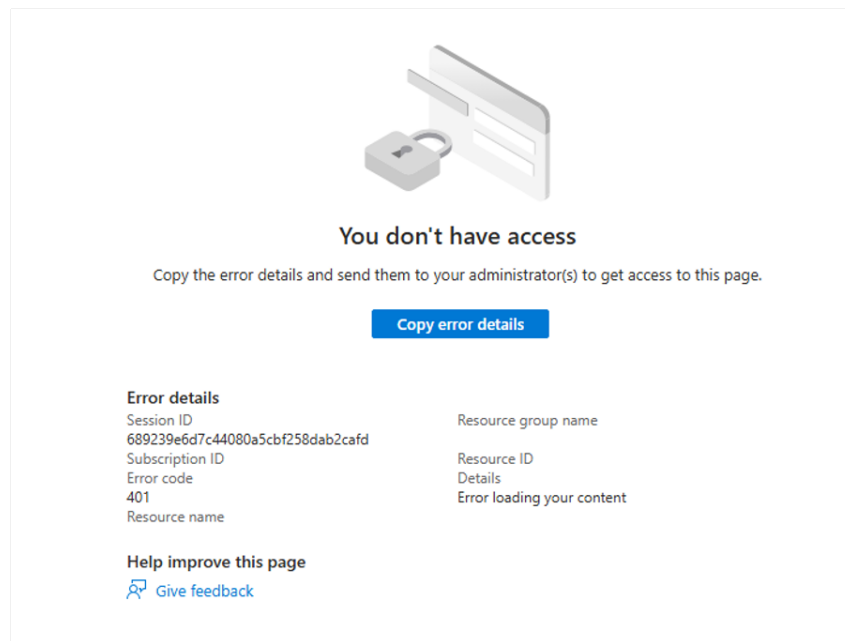


Figura 1: Erro 401 (Unauthorized) no portal do Azure ao tentar registar a aplicação no *tenant* da UAlg.

Solução de contingência: *Magic Links*. Face à impossibilidade de integração OAuth/-SAML direta sem a intervenção demorada da administração de TI da UAlg, a equipa optou por uma estratégia de autenticação *passwordless* (*magic links* através do *Email Provider* do NextAuth). Esta solução justifica-se por:

- **Independência de infraestrutura.** O sistema funciona de forma autónoma, sem depender de autorizações de terceiros (Serviços de Informática) para operar.
- **Exclusividade e validação de domínio.** O *login* é intercetado e o domínio do email validado por *regex*, garantindo que apenas endereços terminados em *@ualg.pt* recebem o link de acesso e conseguem criar conta.
- **Segurança.** Mantém-se o princípio de não armazenar *hashes* de *passwords* (a tabela *User* não possui campo *password*), mitigando o risco de fugas de dados.
- **Experiência de utilizador.** O fluxo atua como um SSO simplificado: o utilizador não precisa de memorizar uma nova *password*, bastando clicar no link seguro enviado para a caixa de correio institucional.

7 Modelo de Dados

A persistência é definida no *schema* Prisma. As entidades dividem-se em dois grupos: as relativas à autenticação (*User*, *Account*, *Session*, *VerificationToken*, herdadas do modelo do NextAuth.js) e as do domínio Spot-On.

7.1 Entidades de Domínio

Entidade	Descrição
User	Utilizador autenticado. Inclui pontos acumulados e prazo de banimento, se aplicável.
FloorPlan	Planta de um piso da biblioteca; serve de <i>canvas</i> sobre o qual os espaços são posicionados.
Space	Espaço de estudo (mesa individual ou sala de grupo). Inclui geometria (<i>points</i> SVG), capacidade, atributos (tomada, computador, quadro) e o <i>token</i> QR rotativo.
StudySession	Sessão de estudo num espaço, com anfitrião, hora prevista de fim e estado (ativa, concluída, expirada).
UserOnStudySession	Tabela-pivô que regista a participação de utilizadores em sessões, com o estado do convite.
Report	<i>Report</i> submetido por um utilizador sobre uma sessão, com janela de confirmação de 10 minutos.
ReportConfirmation	Confirmação de um <i>report</i> por um terceiro utilizador.
Notification	Notificação in-app dirigida a um utilizador.
Badge	Insígnia que pode ser atribuída (mensal, anual ou permanente).
UserBadge	Tabela-pivô que regista a atribuição de <i>badges</i> a utilizadores.

Tabela 5: Entidades do modelo de domínio.

7.2 Diagrama do Modelo de Dados

[Diagrama do modelo de dados a inserir]

7.3 Decisões de Modelação Relevantes

- **Geometria dos espaços armazenada como *string* SVG.** O atributo *points* de *Space* segue o formato "*x1,y1 x2,y2 ...*", compatível com o elemento `<polygon>` do SVG. Esta decisão evita a sobrecarga de uma tabela auxiliar de vértices e permite *rendering* direto na interface.
- **Token QR rotativo.** O atributo *currentQrToken* é único e atualizado periodicamente por uma rotina do servidor, impedindo que um QR fotografado uma única vez possa ser reutilizado posteriormente.
- **Janela de confirmação de *reports*.** O campo *timeToConfirm* usa o valor por omissão gerado na base de dados (`now() + interval '10 minutes'`), garantindo coerência mesmo perante dessincronização horária no *frontend*.
- **Tabelas-pivô explícitas.** *UserOnStudySession*, *ReportConfirmation* e *UserBadge* são modeladas como entidades próprias por carregarem informação adicional (estado do convite, *timestamp* de confirmação, data de atribuição).
- **Banimento como *DateTime*.** *User.bannedUntil* guarda diretamente o instante até ao qual o banimento é válido, eliminando a necessidade de um *flag* *vs.* data calculada.

7.4 Excerto do *Schema* Prisma

A título ilustrativo, apresenta-se a definição da entidade central *Space*:

```
model Space {
  id          String      @id @default(cuid())
  floorPlanId String
  name        String
  points       String      // pontos SVG do polígono
  capacity     Int
  currentQrToken String    @unique
  hasPowerOutlet Boolean   @default(false)
  hasComputer  Boolean   @default(false)
  hasInteractiveBoard Boolean @default(false)
  type         SpaceType  @default(INDIVIDUAL_DESK)
  shape        SpaceShape?

  floorPlan FloorPlan @relation(fields: [floorPlanId], references: [id])
  sessions  StudySession[]
}
```

8 Backend e API

O *backend* é exposto através das *API routes* do Next.js, sob `/api/*`. A API segue um estilo **REST**, com *endpoints* organizados por recurso (`/api/spaces`, `/api/sessions`, `/api/reports`, etc.). Cada *endpoint* valida o utilizador autenticado, valida os dados de entrada e delega o acesso a dados ao Prisma *Client*.

A persistência é totalmente versionada por migrações Prisma. A cada alteração do *schema*, é gerada uma nova migração SQL com `npx prisma migrate dev`, ficando registada no diretório `prisma/migrations`. Em produção, as migrações são aplicadas automaticamente no *deploy* via `prisma migrate deploy`.

9 Controlo de Versões

Todo o desenvolvimento é gerido em Git, com repositório alojado no GitHub e partilhado em modo de leitura com os docentes. Cada elemento da equipa contribui sob a sua identidade pessoal, garantindo que o histórico reflete fielmente a contribuição individual.

Fluxo adotado. Foi seguido um modelo simplificado baseado em *feature branches*:

1. Cada *user story* ou tarefa origina um *branch* dedicado (`feature/<descrição>`);
2. A integração no *branch* principal (`main`) ocorre exclusivamente via *pull request*;
3. Cada *pull request* requer revisão por outro elemento da equipa antes de *merge*;
4. A *pipeline* de CI deve passar (*lint*, testes, *build*) como pré-condição.

10 Gestão de Projeto – SCRUM

A gestão do projeto seguiu a metodologia SCRUM, suportada pela plataforma Jira. Definiu-se um *product backlog* composto por *user stories* com critérios de aceitação claros. As *stories* foram refinadas e priorizadas no início de cada *sprint*.

Cerimónias. Cada *sprint* (com duração de **[X]** semanas) inclui:

- ***Sprint planning*** – seleção das *stories* a incluir no *sprint*;
- ***Daily*** – ponto de situação curto entre os elementos;
- ***Sprint review*** – demonstração do incremento;
- ***Retrospectiva*** – análise do que correu bem, do que deve melhorar e ações a tomar.

Evidências. O Jira mantém o histórico de *sprints*, cartões movidos entre estados (*To Do*, *In Progress*, *In Review*, *Done*), e os *burndown charts* associados.

11 Testes e Integração Contínua

11.1 Testes Automatizados

A estratégia de testes assenta em **Jest**, executado quer localmente quer na *pipeline* de CI. Os testes cobrem:

- **Testes unitários** – funções de lógica de negócio (e.g., cálculo de pontos, validação de *tokens*, regras de elegibilidade para sessão);
- **Testes de integração** – *endpoints* da API contra uma base de dados de teste, validando o comportamento ponta-a-ponta da camada de servidor.

11.2 Pipeline de CI

A *pipeline* é definida em `.github/workflows` e dispara a cada *push* ou *pull request* para `main`. Os passos são:

1. *Checkout* do repositório;
2. Instalação de dependências (`npm ci`);
3. Análise estática de código (`lint`);
4. Execução dos testes Jest;
5. *Build* de produção (`next build`).

A passagem destes passos é uma pré-condição obrigatória para a aprovação dos *pull requests*.

12 Deployment

12.1 Execução Local

A execução local recorre a **Docker Compose**, com dois serviços: a aplicação Next.js e a instância de PostgreSQL. Após clonar o repositório, a aplicação é colocada em execução com um único comando:

```
docker compose up --build
```

As migrações da base de dados são aplicadas automaticamente no arranque do *container*.

12.2 Ambiente de Produção

Para o ambiente de produção, recorre-se à plataforma **Railway**, que oferece (i) *deploy* contínuo a partir do repositório Git, (ii) base de dados PostgreSQL gerida com *backups* automáticos, e (iii) gestão de variáveis de ambiente (`DATABASE_URL`, segredos do NextAuth, etc.).

13 Conclusão

O Spot-On materializa uma solução completa para um problema concreto vivido diariamente na biblioteca da Universidade do Algarve. Do ponto de vista de engenharia de software, o projeto permitiu aplicar boas práticas em todas as fases: levantamento e priorização de requisitos sob SCRUM, desenho de uma arquitetura clara e justificada, implementação com tecnologias modernas e tipadas, persistência versionada com migrações, testes automatizados, *deployment* reproduzível e Integração Contínua.

Como trabalho futuro, identificam-se eixos como (i) a integração com sistemas de *notification push* para alertas em tempo real, (ii) a exposição de estatísticas agregadas de utilização para apoio à gestão da biblioteca e (iii) o alargamento do sistema a outros campi da Universidade.